CSOPESY Process Scheduler — Layout & Build Prompt

Oct 26, 2026

Project Overview

This is the single build reference for the CSOPESY Major Output: a CLI-based process scheduler/multiplexer (the "CSOPESY Emulator") written in C++, modeled on Linux `screen`, SLURM-style batch scripts, and PowerShell. It turns every requirement in the handout into (1) a concrete project layout and (2) one consolidated build prompt that can be handed to an AI coding agent or followed step by step directly — so nothing gets missed before the Week 8 black-box quiz, where only `config.txt` may change between test cases and recompiling is not allowed.

Proposed Project Layout

A single-binary C++17 project split by responsibility, so the scheduler, CLI parser, and instruction interpreter can be built and tested in isolation:

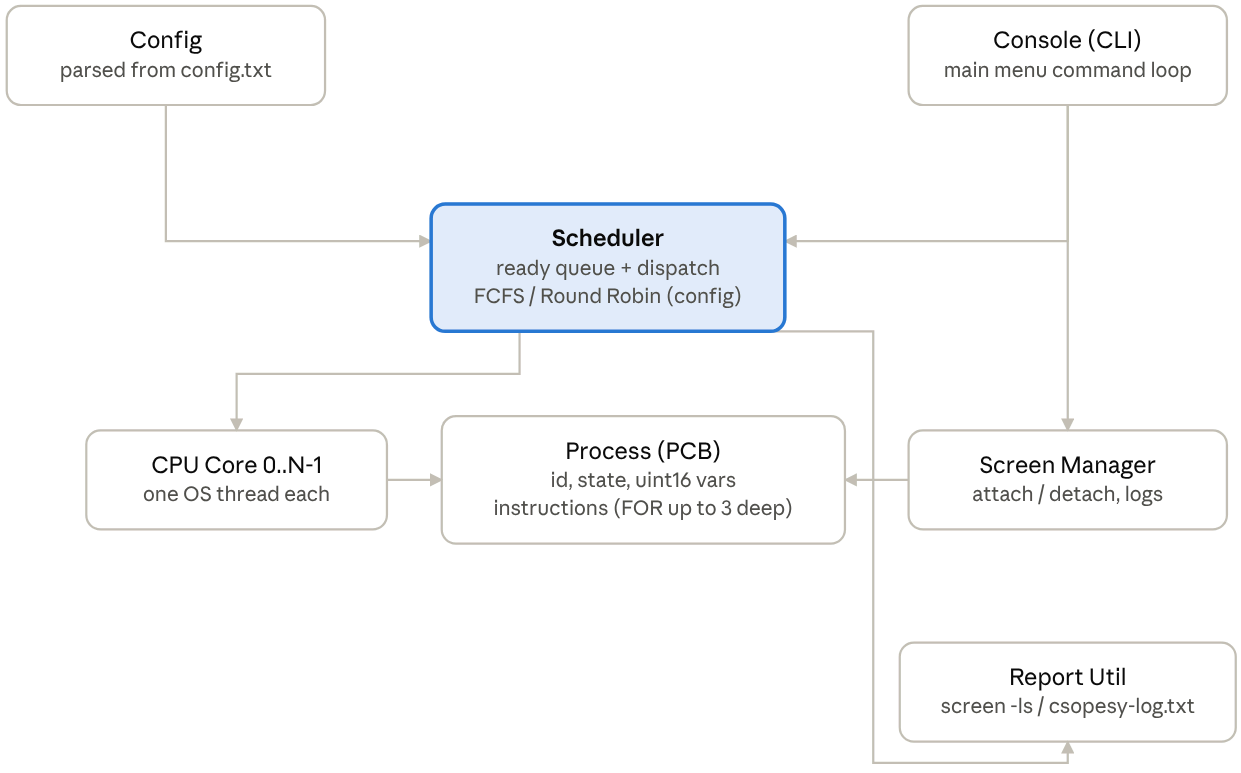
```
csopesy/
├── config.txt
├── CMakeLists.txt (or Makefile)
└── src/
    ├── main.cpp                    # entry point, owns the main menu loop
    ├── cli/
    |   ├── Console.h / .cpp        # main menu command dispatch, banner,
prompt loop
    |   └── CommandParser.h / .cpp  # tokenizes and validates typed commands
    ├── config/
    |   └── Config.h / .cpp         # parses config.txt, holds the global
parameters
    ├── process/
    |   ├── Process.h / .cpp        # PCB: id, name, state, variable table,
instruction list, log
    |   ├── Instruction.h / .cpp    # PRINT / DECLARE / ADD / SUBTRACT /
SLEEP / FOR
    |   └── InstructionGenerator.h / .cpp  # randomized instruction
generation within min-ins/max-ins
```

```
    ├── scheduler/
    |   ├── Scheduler.h / .cpp       # ready queue, dispatch loop, FCFS/RR
strategy switch
    |   ├── Core.h / .cpp            # one simulated CPU core, runs on its
own thread
    |   └── Clock.h / .cpp           # the global CPU tick counter
    ├── screen/
    |   └── ScreenManager.h / .cpp   # screen -s / -r / -ls, attach/detach,
process-smi rendering
    └── report/
        └── ReportUtil.h / .cpp      # builds the screen -ls text, writes
csopesy-log.txt
```

One OS thread per simulated core, pulling from a single mutex-protected ready queue, keeps FCFS and Round Robin swappable at the `Scheduler` layer without touching the CLI or process model.

Core Architecture

One scheduler coordinates every core, screen, and report



architecture · Config + Console drive Scheduler, Cores, Screens, Report

Config feeds the Scheduler, which dispatches ready Processes onto free Cores, while the Console drives both the Screen Manager and the reporting path.

Component	Key fields / responsibility
Config	numCpu, scheduler (fcfs/rr), quantumCycles, batchProcessFreq, minIns, maxIns, delaysPerExec
Process (PCB)	id, name, state (Ready/Running/Sleeping/Finished), coreId, programCounter, totalInstructions, variables (map<string,uint16_t>), timestamped print log
Instruction	type (Print/Declare/Add/Subtract/Sleep/For), operands; For also holds a nested instruction list + repeat count
Scheduler	ready queue, vector of Cores, active strategy (FCFS/RR), per-core quantum counters
Core	coreId, pointer to its current Process, ticks run on that process

Main Console Commands

Command	Syntax	Behavior
initialize	`initialize`	Must run first (the only other command recognized before it is `exit`). Reads `config.txt`, builds the `Config`, and spins up the `Scheduler` with `num-cpu` cores.
exit	`exit`	Terminates the console. (Different from the in-screen `exit` below.)
screen (create)	`screen -s <name>`	Creates a new `Process` named `<name>`, generates its randomized instruction list (length between `min-ins` / `max-ins`), enqueues it, and attaches the console to its screen.

Command	Syntax	Behavior
screen (reattach)	`screen -r <name>`	Reattaches to an existing process's screen. Prints `Process <name> not found.` if it doesn't exist or has already finished.
screen (list)	`screen -ls`	Prints CPU utilization %, cores used, cores available, then a summary of Running and Finished processes (core, timestamp, progress `x / total`).
scheduler-start	`scheduler-start`	Begins auto-generating a new dummy process every `batch-process-freq` CPU ticks, feeding the ready queue as long as a core is free. (Formerly `scheduler-test`.)
scheduler-stop	`scheduler-stop`	Stops generating new dummy processes; already-running ones continue to completion.
report-util	`report-util`	Writes the same content as `screen -ls` to `csopesy-log.txt` in the working directory and confirms with `Report generated at C:/csopesy-log.txt!`

Process Screen & process-smi

Attaching to a process (`screen -s` / `screen -r`) clears the console and swaps in a per-process prompt with two commands:

Command	Behavior
process-smi	Prints the process name, ID, and its full timestamped log so far (core + timestamp + message). Also prints the current instruction line and total lines of code. If the process has finished, appends `Finished!` after that block.
exit	Returns to the main menu. Does not terminate the process.

A process keeps running in the background whether or not anyone is attached; `process-smi` only reports its latest state. Once a process finishes, its screen can no longer be entered via `screen -r`.

Instruction Set Semantics

Each process's instruction list is pre-generated at creation time — never typed by the user — and runs against process-local `uint16_t` variables that persist for the process's whole lifetime, clamped to `[0, 65535]`.

Instruction	Syntax	Semantics
PRINT	`PRINT(msg)`	Writes `msg` to the process's log, visible only via `process-smi` while attached. Default message unless a test case says otherwise: `"Hello world from <process_name>!"`. Can interpolate exactly one variable, e.g. `PRINT("Value from: " + x)`.
DECLARE	`DECLARE(var, value)`	Declares `var` as a `uint16_t` initialized to `value`.
ADD	`ADD(var1, var2/value, var3/value)`	`var1 = var2 + var3`; operands 2–3 may be a variable or a literal. Undeclared variables auto-create at 0 before the op runs. Result clamps to the `uint16_t` range.
SUBTRACT	`SUBTRACT(var1, var2/value, var3/value)`	`var1 = var2 - var3`, same auto-declare and clamping rules as ADD (clamps at 0 on underflow, never wraps).
SLEEP	`SLEEP(X)`	Busy-waits the process for `X` (`uint8_t`) CPU ticks, relinquishing the core for that time.
FOR	`FOR([instructions], repeats)`	Repeats the given instruction block `repeats` times; nestable up to 3 levels deep.

config.txt Parameters

Space-separated, read once by `initialize`:

Parameter	Range	Effect
num-cpu	[1, 128]	Number of simulated CPU cores.
scheduler	`"fcfs"` or `"rr"`	Scheduling algorithm.
quantum-cycles	[1, 2^32]	Round-robin time slice per core; ignored under FCFS.
batch-process-freq	[1, 2^32]	CPU ticks between each new dummy process spawned by `scheduler-start` (1 = a new process every tick).
min-ins / max-ins	[1, 2^32] each	Range for the number of instructions randomly generated per process.
delays-per-exec	[0, 2^32]	Busy-wait delay, in CPU ticks, before executing each instruction; 0 = one instruction per tick.

Default sample `config.txt`:

```
num-cpu 4
scheduler "rr"
quantum-cycles 5
batch-process-freq 1
min-ins 1000
max-ins 2000
delay-per-exec 0
```

Scheduler Algorithms — FCFS & Round Robin

The scheduler runs continuously once `initialize` completes, one dispatch loop per core, picked by the `scheduler` parameter:

- **FCFS (First-Come-First-Served):** each process runs to completion, or until it hits SLEEP, on whichever core frees up next, in ready-queue order. No preemption — `quantum-cycles` has no effect.

- **Round Robin:** each core runs its current process for up to `quantum-cycles` ticks, then preempts it back onto the end of the ready queue even if unfinished, and pulls the next ready process.

A process's lifecycle is the same under both algorithms: `New → Ready → Running → Finished`, with two extra edges — `Running → Ready` (preempted at the end of a Round Robin quantum) and `Running → Sleeping → Ready` (a SLEEP(X) instruction parks the process for X ticks without holding a core, then returns it to the ready queue). The CPU tick itself is just an integer incremented once per loop pass while the console is alive — not wall-clock time.

Reporting — screen -ls & report-util

`screen -ls` and `report-util` render identical content — one to the console, one appended to `csopesy-log.txt` — so one formatter function plus two thin callers covers both commands. Expected shape:

```
CPU utilization: 100%
Cores used: 4
Cores available: 0

Running processes:
process05   (01/18/2024 09:15:22AM)   Core: 0   1235 / 5876
process06   (01/18/2024 09:17:22AM)   Core: 1   3 / 5876
process07   (01/18/2024 09:17:45AM)   Core: 2   9 / 1000
process08   (01/18/2024 09:18:58AM)   Core: 3   12 / 80

Finished processes:
process01   (01/18/2024 09:00:21AM)   Finished   5876 / 5876
process02   (01/18/2024 09:00:22AM)   Finished   5876 / 5876
process03   (01/18/2024 09:00:42AM)   Finished   1000 / 1000
process04   (01/18/2024 09:00:53AM)   Finished   80 / 80
```

`report-util` writes this same block to `csopesy-log.txt` and confirms with `Report generated at C:/csopesy-log.txt!`. Both finished and currently running processes must appear, for debugging/validation purposes.

Full Build Prompt

Everything above consolidated into one prompt — copy this whole block to hand off to an AI coding agent, or follow it directly section by section.

```
# CSOPESY Emulator — Build Prompt

You are implementing a C++17 CLI program called the "CSOPESY Emulator": a
process multiplexer and command-line scheduler. Follow every rule below
exactly — grading is a black-box, timed quiz where only config.txt may be
edited between test cases, with no recompiling, so behavior must match these
specs precisely rather than approximately.

## 1. Startup & main menu
- On launch, print the ASCII banner (title, developer names, last-updated
date) and a `root:\>` prompt.
- Recognize only `initialize` and `exit` before `initialize` has run. Any
other command prints an error and re-prompts.
- `initialize` reads config.txt (space-separated key/value pairs: num-cpu,
scheduler, quantum-cycles, batch-process-freq, min-ins, max-ins, delays-per-
exec), validates every value against its stated range, and constructs the
Scheduler with that many Cores.
- After `initialize`, recognize: `screen`, `scheduler-start`, `scheduler-
stop`, `report-util`, `exit`.

## 2. screen command
- `screen -s <name>`: create a new Process `<name>` with a randomized
instruction list (length uniformly random between min-ins and max-ins),
enqueue it on the ready queue, clear the console, and attach to its screen.
- `screen -r <name>`: reattach to an existing, unfinished process's screen.
If not found or finished, print exactly `Process <name> not found.` and stay
on the main menu.
- `screen -ls`: print CPU utilization %, cores used, cores available, then
"Running processes:" and "Finished processes:" sections.
- Inside a process's screen: `process-smi` shows name, ID, full print log
(core + timestamp + message per line), current instruction line, total lines
of code, and `Finished!` if done; `exit` returns to the main menu without
killing the process.

## 3. Instruction set
Implement exactly these six instruction types, generated randomly (never
typed by the user) within each process's instruction count:
- `PRINT(msg)` — log to the process's buffer; visible only through process-
```

```
smi. Default message `"Hello world from <process_name>!"` unless a test case
specifies otherwise. Supports exactly one interpolated variable.
- `DECLARE(var, value)` — create a uint16_t variable.
- `ADD(var1, var2/value, var3/value)` — var1 = var2 + var3, clamped to [0,
65535]; auto-declare undeclared operands at 0.
- `SUBTRACT(var1, var2/value, var3/value)` — same rules as ADD, clamped at 0
on underflow (never wraps).
- `SLEEP(X)` (uint8_t X) — park the process off the core for X CPU ticks,
then return it to Ready.
- `FOR([instructions], repeats)` — repeat a nested instruction block
`repeats` times; support nesting up to exactly 3 levels.
Variables are process-local and persist until the process finishes.

## 4. Scheduler
- Build one thread per num-cpu core, pulling from a single mutex/condvar-
protected ready queue.
- scheduler == "fcfs": run each process to completion (or to its next SLEEP)
with no preemption; quantum-cycles has no effect.
- scheduler == "rr": preempt the running process back to the end of the ready
queue after quantum-cycles ticks if it hasn't finished.
- The CPU tick is a plain incrementing counter, advanced once per scheduler
loop pass, independent of wall-clock time.
- delays-per-exec ticks of busy-wait happen before each instruction executes
(0 = one instruction per tick).

## 5. scheduler-start / scheduler-stop
- scheduler-start: every batch-process-freq CPU ticks, spawn one new dummy
process with a human-readable, unique name (e.g. p01, p02, …) and enqueue it,
as long as the generator is running.
- scheduler-stop: stop spawning new processes; existing ones keep running to
completion.

## 6. Reporting
- Implement one formatter that renders CPU utilization %, cores
used/available, and the Running/Finished process tables.
- screen -ls prints it to the console; report-util appends it to csopesy-
log.txt and prints `Report generated at C:/csopesy-log.txt!`. Both running
and finished processes must always be listed, for grading/debugging.

## 7. Non-functional constraints
- No command outside initialize/exit may run before initialize completes.
- Keep all behavior driven by config.txt, since the quiz format forbids
recompiling between test cases — only parameters change.
```

```
- Prefer clear separation (CLI parsing, process model, scheduler, screen
manager, reporting) over one large file, so a single quiz question's tweak
doesn't risk the rest.

Deliver working, compiled C++17 source implementing the above, plus a short
note on any assumption made where the spec was ambiguous.
```

Suggested Implementation Order

1. Config loader — parse config.txt into a Config struct, validate every range, unit-test against the sample file.
2. CLI shell — main menu loop recognizing only initialize/exit pre-initialize, prints the banner.
3. Process model — Process/PCB struct, Instruction base + the 6 instruction types, variable table with clamped uint16_t arithmetic.
4. Instruction generator — randomized instruction lists within min-ins/max-ins, nested FOR up to 3 levels.
5. Single-core FCFS scheduler — get one process running start-to-finish on one core before adding concurrency.
6. Multi-core + threads — scale to num-cpu cores, each on its own thread, shared ready queue behind a mutex/condvar.
7. Round robin — add quantum-cycles preemption and requeueing; verify against FCFS by toggling `scheduler` in config.txt.
8. screen command — -s/-r/-ls, process-smi rendering, attach/detach, "Process <name> not found." handling.
9. scheduler-start / scheduler-stop — batch-process-freq-driven generation of human-readable process names.
10. report-util — reuse the screen -ls formatter, write to csopesy-log.txt.
11. Timing knobs — delays-per-exec busy-wait, SLEEP's tick-based wake-up; confirm the CPU tick increments match the pseudocode.
12. Dry run against the quiz format — practice changing only config.txt between runs, without recompiling, and recording one continuous, unedited .mp4 end to end.

Edge Cases & Clarifications to Confirm

Worth settling before quiz day, since grading is black-box and exact output matters:

- [ ] Exact timestamp format expected in logs/reports (`MM/DD/YYYY hh:mm:ssAM/PM`) — match the sample exactly.
- [ ] Whether PRINT's single-variable interpolation needs the `"text" + var` concatenation syntax shown in the spec, or just a bare variable name.
- [ ] Whether finished processes stay listed in `screen -ls` / `report-util` forever, or only for the current run.
- [ ] Whether `scheduler-start` naming (`p01, p02, … p1240`) must follow that exact zero-padded/sequential scheme or just be unique and human-readable.
- [ ] How ties are broken when multiple cores free up in the same tick under FCFS.
- [ ] Whether `delays-per-exec` applies between every instruction, including inside FOR loop bodies, or only at the top level.
- [ ] What a `uint16_t` SUBTRACT underflow should clamp to — confirmed as 0 above, but worth double-checking against any released test case.